ESP32-Based Predictive Motor Maintenance

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in the Department of Electronics and Communication Engineering

Embedded Systems 25EC2206E

Submitted by

2510030354: Mohammad Reyhan

2510030071: Syed Faizan

2510030428: Amaan Pasha Shaikh

Under the guidance of

Dr.Madhu Oruganti



Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

Abstract

Electric motors support pumps, conveyors, ventilation systems and many other machines. Unexpected motor failure interrupts operation and can increase repair costs. Vibration monitoring offers a way to observe changes in mechanical behaviour while a motor is running. This project presents an ESP32-based monitoring approach that uses an ADXL335 accelerometer to acquire vibration signals and identify unusual vibration through a baseline or threshold comparison.

The ADXL335 provides analog acceleration signals. The ESP32 samples these signals through its analog-to-digital converter and supports embedded processing in C/C++ using the Arduino IDE. The monitoring method groups samples into windows, removes the steady component and evaluates amplitude features such as root mean square vibration and peak amplitude. A threshold comparison produces a normal or anomaly indication that can be observed through the Serial Monitor or Serial Plotter.

The project focuses on a low-cost, explainable approach to motor condition monitoring. The detection method uses explicit rules rather than a trained machine-learning model. An anomaly indicates a departure from the selected operating baseline and can prompt inspection. It does not by itself identify the failed component or estimate the remaining useful life of the motor.

A structured evaluation can compare baseline operation with controlled vibration changes and assess false alarms, detection rate and response time. Future development can add temperature sensing, improved filtering, condition-specific calibration and wireless monitoring. The embedded design provides a foundation for maintenance decisions supported by measured motor condition.

Keywords: ESP32, ADXL335, motor condition monitoring, vibration analysis, threshold detection, predictive maintenance.

List of Figures

Figure 1. Vibration monitoring signal path and status output

Figure 2. Proposed validity and decision-state flow

Figure 3. Illustrative normal and abnormal vibration waveforms

List of Tables

Table 1. Requirements and verification links
Table 2. Hardware components and system roles
Table 3. Software tools and functions
Table 4. Sensor selection checks
Table 5. Mounting record fields
Table 6. Functional connection plan
Table 7. Calibration layers
Table 8. Illustrative acquisition-window calculations
Table 9. Processing operations and their roles
Table 10. Illustrative feature verification cases
Table 11. Threshold configuration record
Table 12. Proposed firmware responsibilities
Table 13. Illustrative resource calculations
Table 14. Proposed feature-record fields
Table 15. Quality checks and interpretation
Table 16. Proposed experimental test matrix
Table 17. Evaluation measures and recording method
Table 18. Commissioning evidence checklist

Table 19. Variables in a comparative trial
Table 20. Proposed label structure
Table 21. Proposed robustness cases
Table 22. Timing endpoints to record
Table 23. Proposed integration tests
Table 24. Suggested run evidence package
Table 25. Troubleshooting paths
Table 26. Project scope and interpretation
Table 27. Hypothetical scoring example
Table 28. Proposed operator interpretation guide
Table 29. Development stages and evidence gates
Table 30. Demonstration review points
Table 31. Glossary of report terms

Table of Contents

Introduction

- Design requirements and project boundaries
- Related monitoring approaches and design rationale

Methodology

- System architecture
- Hardware and sensor integration
- Software environment
- Signal acquisition and feature extraction
- Baseline and detection logic
- Vibration signals and mechanical interpretation
- ADXL335 interface and sensor selection
- Mechanical mounting and axis management
- Electrical integration and power arrangement
- ADC configuration and calibration strategy
- Sampling windows and response-time budgets
- Filtering and the retained vibration band
- Worked example of amplitude feature calculation
- Threshold selection and operating-condition control
- Decision states and warning recovery
- Firmware modules and information flow
- Timing buffers and resource estimates
- Serial records and data dictionary
- Data-quality checks and exceptional conditions

Experiments

- Setup and test procedure

TABLE OF CONTENTS CONTINUED

- Evaluation measures
- Commissioning and calibration protocol
- Controlled experimental design
- Labels and separation of evaluation data
- Robustness and negative-condition tests
- Response-time measurement procedure
- Integration tests and requirements traceability
- Reproducibility and evidence management
- Troubleshooting and error isolation

Results

- System outputs and waveform interpretation
- Scope and limitations
- Illustrative detection-score calculation
- Interpreting trends and responding to warnings
- Limitations and staged development

Conclusion and Future Work

References

Appendix A Processing pseudocode

Appendix B Demonstration and operating procedure

Appendix C Glossary and configuration terminology

ESP32-Based Predictive Motor Maintenance

1. Introduction

Electric motors convert electrical energy into mechanical motion and are central to many everyday and industrial systems. A motor may drive a pump, move a conveyor or circulate air in a ventilation unit. When it stops unexpectedly, the connected process may also stop. The consequences can include lost operating time, repair work and damage to connected equipment. Monitoring a motor while it operates helps maintenance personnel recognise changes before a minor problem becomes a more serious interruption.

Maintenance strategies differ in the information used to decide when to act. Reactive maintenance begins after equipment fails or becomes unusable. Scheduled preventive maintenance follows an interval, even when the actual condition of a component may vary. Condition monitoring instead observes quantities associated with equipment behaviour. Predictive maintenance can use these observations and their trends to support earlier intervention. The value of such monitoring depends on whether the signal is reliable and whether the warning leads to a useful inspection or maintenance decision.

Vibration is a useful signal because a rotating machine produces mechanical motion that changes with its operating condition. A change in mounting, loading or mechanical behaviour can alter the measured vibration. However, a larger signal is not sufficient evidence of a particular defect. Speed changes, environmental vibration and sensor attachment can also influence a reading. A monitoring system therefore needs a repeatable measurement arrangement and a baseline associated with the condition under which the motor normally runs.

This project uses an ADXL335 analog accelerometer with an ESP32 development board. The accelerometer senses motor movement and the ESP32 acquires the analog signal through its ADC. The embedded program provides a path from measurement to a condition indication. The Arduino IDE supports development in embedded C/C++, while the Serial Monitor and Serial Plotter allow the operator to observe readings and signal behaviour. These tools make the sensing and decision process visible during a classroom or laboratory demonstration [1].

The main objective is to explain and apply threshold-based vibration anomaly detection using accessible hardware. The monitoring loop samples vibration, processes a group of readings, compares a feature with a reference and reports the resulting status. Root mean square vibration and peak amplitude are useful candidate features because they summarise overall amplitude and short excursions. The comparison remains explicit, so an operator can relate a warning to the threshold and the observed input signal.

The project scope is motor condition monitoring with an embedded controller. Wireless dashboards, cloud communication and machine learning are extensions rather than requirements of the current monitoring loop. Similarly, an anomaly warning does not constitute a fault diagnosis or a remaining-life prediction. The term predictive maintenance describes the intended maintenance use: observing warning signals early enough to support action before a failure. Establishing that practical benefit requires repeated tests under known operating conditions.

The report explains the architecture, processing method and test procedure, then discusses outputs, limitations and future work.

a. Design requirements and project boundaries

A useful monitoring project begins with a decision that the system is expected to support. Here the decision is whether a change in measured motor vibration deserves attention. The requirement is therefore a repeatable measurement and an understandable warning. A controller that prints rapidly changing numbers without an operating reference would demonstrate acquisition, but it would provide little basis for deciding when to inspect the motor.

The functional requirements cover acquisition, processing and reporting. Acquisition must identify the selected analog channels and preserve the timing needed to interpret a sample window. Processing must apply the same feature definitions during baseline collection and later monitoring. Reporting must associate a status with its feature values and data-quality state. These requirements are separable so that a failure in one stage can be investigated without guessing about the entire system.

A threshold is a configuration value associated with an operating condition. Changing the sensor position, measurement units, sample rate or filtering changes the context in which that threshold was selected. The report therefore treats configuration management as part of measurement integrity. A test is repeatable only when another operator can recover the relevant settings and reproduce the physical arrangement.

Several desirable outcomes remain outside the current verification boundary. The system does not establish the identity of a damaged bearing, a certified vibration severity class or an estimated date of motor failure. It also does not implement an automatic safety shutdown. These capabilities require additional engineering and evidence. Keeping the decision boundary explicit makes an anomaly warning useful without asking it to support conclusions that the measurement cannot justify.

Table 1. Requirements and verification links

Requirement	Evidence needed
Acquire vibration	Recorded channel mapping and samples with timing.
Calculate comparable features	Known-input calculations and repeatable baseline logs.
Identify an exceedance	Logged feature, threshold and decision for the same window.
Report invalid input	Controlled connection or clipping test where supported.
Reproduce a test	Saved firmware, settings and mounting record.

b. Related monitoring approaches and design rationale

Motor maintenance can draw on several kinds of information. Vibration describes mechanical movement at the sensor location. Temperature describes thermal behaviour, which may change more slowly. Current measurements can reveal changes in electrical loading. A speed measurement helps distinguish a change in operating state from a change at a constant state. NI discusses these complementary sensing approaches in its condition-monitoring overview [5]. The selection should follow the engineering question, rather than the availability of a particular sensor.

Amplitude monitoring provides an accessible entry point because it reduces a window to quantities that are easy to display and compare. The limitation is loss of detail. Two waveforms with different shapes can have similar RMS values, while a single impact can dominate a peak feature. This project uses amplitude features to indicate a change and retains the waveform as supporting context. It does not infer a unique defect from a single scalar value.

Frequency analysis can separate periodic components and help relate them to rotation or repeated mechanical events. Such analysis requires a known sample rate, sufficient bandwidth and suitable interpretation. Analog Devices relates vibration sensor bandwidth and noise to the fault phenomena that a monitoring system can observe [4]. That relationship explains why a low-cost demonstration should state its measurement band before discussing advanced fault diagnosis.

Rule-based detection makes the selected condition explicit: a value crosses a configured limit. A trained classifier instead depends on its training data, labels and validation procedure. Neither method removes the need for reliable measurement. A sophisticated classifier trained on inconsistent mounting or incorrectly labelled runs may learn irrelevant differences. The threshold method is appropriate here because it exposes the connection between the input, feature and decision during project review.

The design rationale is to develop a complete local measurement loop before extending its reach. Wireless transmission improves accessibility to information, but it does not correct a poor sensor attachment or an incorrect feature calculation. Cloud storage makes long records easier to retain, but the record still needs units and configuration metadata. The proposed development order is therefore sensing, repeatability, decision validation and then remote access. Each stage adds a capability that can be evaluated independently.

2. METHODOLOGY

a. System architecture

The monitoring system follows a signal path from physical movement to a reported condition. The motor provides the vibration input. A securely attached ADXL335 converts acceleration into analog voltages, and the ESP32 converts the selected channels into digital samples. Processing removes the steady signal component and calculates amplitude features. A baseline or threshold comparison then produces a condition indication. Figure 1 shows this functional sequence. The precise firmware settings remain dependent on the selected board and motor.

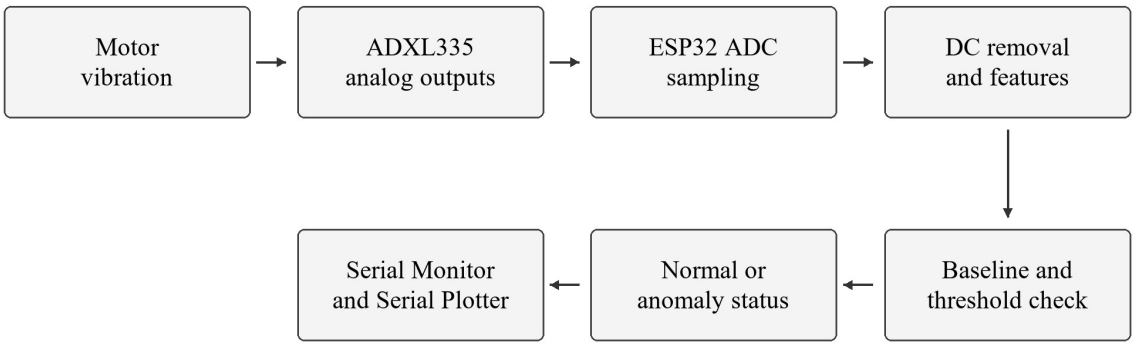


Figure 1. Vibration monitoring signal path and status output

The acquisition and decision steps run locally on the ESP32. The computer serves as a programming and observation interface rather than a required cloud-processing service. This arrangement allows the vibration signal and the decision output to be compared directly during demonstration. If a future implementation introduces networking, its timing and resource use should be evaluated separately from the acquisition loop.

b. Hardware and sensor integration

Table 2. Hardware components and system roles

Component	Purpose	System role
ESP32 development board	Acquire analog sensor readings and run the embedded monitoring logic.	Controller
ADXL335 accelerometer	Provide analog acceleration outputs for vibration measurement.	Sensor
Electric motor	Provide the mechanical system whose condition is observed.	Observed asset
Wires, breadboard or connectors	Connect sensor outputs, supply and ground to the controller.	Integration
USB cable and power supply	Program the controller and provide suitable electrical power.	Support
Computer	Observe readings and plots using the Arduino serial tools.	Visualisation

The ADXL335 is a three-axis accelerometer with signal-conditioned analog outputs. Its data sheet specifies a minimum measurement range of ±3 g and a sensor supply range of 1.8 V to 3.6 V [2]. These are sensor specifications, not measured project performance. A breakout board may include additional circuitry, so its supply requirements and output limits should be checked before wiring. Sensor output voltages must remain within the configured ESP32 ADC measurement range.

A suitable connection maps each selected sensor axis to an ADC-capable input on the actual ESP32 board, with a common ground between the sensor and controller. The motor receives power from an appropriate motor supply rather than a microcontroller signal pin. The final GPIO mapping depends on the development board variant. Recording that mapping with the firmware prevents a wiring diagram for one ESP32 family member from being applied to a different board.

Sensor mounting is part of the measurement system. The sensor should remain fixed to the motor housing or a repeatable mounting point, with the axis orientation recorded. A loose sensor or moving cable can introduce a signal unrelated to the motor condition being investigated. Repeated measurements should keep the attachment position, motor support and cable arrangement consistent. The reference condition should be re-established if these factors change.

c. Software environment

Table 3. Software tools and functions

Tool or function	Purpose
Arduino IDE	Edit, build and upload the embedded program.
Embedded C/C++	Implement acquisition, feature calculation and threshold decisions.
ESP32 ADC interface	Read the selected analog input channels.
Serial Monitor	Display readings, feature values and condition status.
Serial Plotter	Observe changes in numerical signals over time.

The software can separate acquisition, feature calculation and reporting into distinct functions. Acquisition collects samples with recorded timing. Processing operates on a completed window, and reporting publishes the features and status. Keeping serial messages outside the timing-critical part of acquisition reduces the chance that slow output changes the effective sampling interval. The firmware should record its configuration so later test runs can be compared consistently.

Espressif documents analogRead as a raw, non-calibrated conversion and analogReadMilliVolts as a calibrated millivolt reading [3]. The chosen interface and ADC configuration should be stated in the implementation record. A raw ADC count is not automatically a physical acceleration value. Conversion to acceleration requires the sensor offset and sensitivity at the actual supply, together with a suitable ADC calibration. Without that calibration, features should be reported in ADC counts or the voltage units actually used.

d. Signal acquisition and feature extraction

A sampling window contains N readings collected at a nominal sample rate fs. Its duration is N divided by fs. Selecting these quantities balances update speed against the stability of the calculated feature. The useful vibration bandwidth must also be compatible with the sensor module, its filtering and the sampling rate. Sampling above twice the highest retained signal frequency is a necessary anti-aliasing condition, with suitable filtering required before conversion. No particular sample rate or window size is assumed for the prototype.

The accelerometer output includes a steady bias and may include a constant contribution from gravity. Calculating vibration amplitude from the uncorrected signal can therefore confuse the steady component with motor vibration. For a simple amplitude-monitoring method, estimate the mean of each acquisition window and subtract it from each sample. Let x_i denote a sample, μ the window mean and d_i the resulting centred sample. Here n = N is the total number of samples. The following definitions apply to one selected axis:

$$\mu = (x_1 + x_2 + \cdots + x_n) / N$$

$$d_i = x_i - \mu$$

$$RMS = \sqrt{[(d_1^2 + d_2^2 + \cdots + d_n^2) / N]}$$

$$Peak = max(|d_1|, |d_2|, \ldots, |d_n|)$$

RMS summarises the amplitude over the full window, while the absolute peak identifies the largest excursion. Both features must use consistent units and the same definition during baseline and test measurements. When multiple axes are acquired, calculate features per axis or specify exactly how the axes are combined. Reporting the selected axis and attachment orientation makes the feature values easier to interpret.

Window-mean removal reduces the steady component but does not provide complete noise filtering or frequency-selective fault analysis. Electrical noise, motor speed changes and clipping can still affect the calculated amplitude. The acquisition should flag disconnected sensors, persistent rail values or missed timing where the implementation can detect them. Such data-quality problems should lead to an invalid-reading indication rather than an unqualified normal status.

e. Baseline and detection logic

Baseline collection uses a stable motor condition considered acceptable for the demonstration. Several consecutive windows are recorded after the motor reaches the selected operating state. Their features establish the normal variation under that condition. The detection threshold is then chosen with reference to this variation and checked against separate trials. A threshold copied from another motor or mounting arrangement can produce misleading warnings.

The core rule compares a current vibration feature with its configured threshold. For example, a centred RMS value above an RMS limit, or a peak value above a peak limit, can produce an anomaly indication. A normal indication means that the selected rule did not detect an exceedance in that window. It does not guarantee that every component is healthy. The limit and units should be displayed or recorded with the test configuration.

Illustrative processing sequence

Initialise the serial interface, ADC inputs and acquisition configuration.

Acquire a complete window of vibration samples with consistent timing.

Check the readings, estimate the mean and calculate centred RMS and peak amplitude.

Compare the features with the selected baseline limits.

Report the feature values and normal, anomaly or invalid-reading status.

Repeat acquisition for the next window.

This sequence explains the rule-based method and is not a claim about a particular uploaded firmware listing. Persistence checks across consecutive windows or separate entry and recovery thresholds can reduce rapid switching near a limit. These refinements should be introduced as explicit configuration choices and assessed for their effect on both false alarms and response time.

f. Vibration signals and mechanical interpretation

A rotating motor produces motion at many locations on its housing and support. The signal measured at one location is the response of the whole mechanical path between the vibration source and the accelerometer. This path includes the motor structure, attachment interface and sensor package. Consequently, the measurement is not a direct photograph of an internal component. It is a response that can change when either the source or the transmission path changes.

Acceleration describes the rate of change of velocity. In a periodic motion, acceleration amplitude depends on both motion amplitude and frequency. A high-frequency movement with small displacement can therefore produce a noticeable acceleration signal. This matters when interpreting a plot: a larger acceleration feature does not automatically mean a proportionally larger shaft displacement. The report keeps the measured quantity as acceleration or ADC amplitude and avoids converting it into displacement or velocity without the required processing.

A stable rotating machine may contain a repeated component related to rotation as well as smaller disturbances. A loose support can introduce additional movement, while an external vibration source can enter through the bench. Startup, shutdown and changing load can produce transients. These descriptions identify possible influences on the signal, not diagnoses from the current prototype. A test label should record which influence the operator intentionally changed.

Sensor orientation also changes the observed projection of motion. A radial axis may respond differently from an axial direction, even during the same run. A three-axis sensor provides the option to compare directions, but sequential acquisition and separate calibration must be considered. A combined magnitude feature can be useful only after its definition is fixed. Combining uncorrected signals can mix the gravitational component with the dynamic response.

The practical interpretation rule is to compare like with like. Keep mounting, axis, speed region and processing settings consistent, and examine departures from a documented baseline. If one of these conditions changes, establish whether the baseline is still appropriate. This approach does not require knowledge of every internal fault mechanism. It requires enough control of the measurement to decide whether the observed change is credible and whether inspection is warranted.

g. ADXL335 interface and sensor selection

The ADXL335 is the sensing element identified in the project presentation. It provides separate analog outputs for the three axes. The interface therefore requires an analog acquisition path rather than a digital bus transaction. At the system level, each output is treated as a voltage containing an offset plus a dynamic component. The offset must be distinguished from the vibration amplitude before window features are interpreted.

The manufacturer specifies a minimum ±3 g measurement range and a sensor supply range of 1.8 V to 3.6 V [2]. These limits describe the sensor component. A purchased module may include a regulator, capacitors or other circuitry that changes how its supply pin should be used. The module schematic or supplier documentation should therefore be checked alongside the component data sheet. A label on a connector is not enough to establish the electrical behaviour of the whole board.

Sensor selection should consider the amplitude and frequency content of the intended demonstration. If the movement exceeds the available range, the signal can clip and the calculated feature can underestimate further changes. If the relevant vibration lies outside the usable bandwidth, an apparently quiet reading may reflect the measurement limit rather than a quiet motor. A later industrial design may require a different sensor even if the basic monitoring logic remains useful.

The sensor output range must be compatible with the selected controller inputs. Supply voltage alone does not establish this compatibility because the ADC range also depends on its configuration and the specific ESP32 device. Before connecting the sensor output, verify the intended operating range and the available measurement headroom. Electrical verification should precede feature calibration so that calibration is not used to hide an input-range problem.

Selection records should identify the actual module, not just the chip family. Photographs of the board, a connector description and any available module schematic help later teams reproduce the setup. If a replacement module is installed, repeat the relevant offset, range and baseline checks. Substituting a similar-looking module without checking its filtering and orientation can change the measured response even when the firmware is unchanged.

Table 4. Sensor selection checks

Check	Reason
Module identity	Distinguish the installed board from the bare sensor specification.
Range and headroom	Recognise clipping and retain usable vibration variation.
Output compatibility	Keep the controller input within its configured range.
Axis orientation	Compare the same direction across repeated runs.

h. Mechanical mounting and axis management

The mounting method should transfer motion from the motor housing to the sensor with as little uncontrolled movement as possible. A board that rocks on a cable can produce a large reading even when the motor condition is unchanged. The attachment therefore needs a defined location, orientation and fastening method. The choice should suit the laboratory rig and should not interfere with moving parts, ventilation or access needed to operate the motor.

A mounting record can use a photograph together with a short written description. Identify a physical reference such as a housing edge so that another operator can locate the same point. Describe which sensor axis points along the shaft and which axes lie across it. The purpose is reproducibility. A precise numerical feature is difficult to interpret when the direction and mechanical coupling are unknown.

Cable routing belongs in the mounting procedure. The wires should have enough strain relief that their movement does not pull the sensor or change the contact pressure. Keep their route consistent between baseline and comparison runs. If the signal changes when a cable is gently repositioned with the motor off, investigate that sensitivity before attributing later changes to motor operation.

Removing and reinstalling the sensor is a useful repeatability test. Compare within-run variation with the variation caused by remounting. If remounting changes the feature more than the intended test condition, the installation procedure needs improvement before a common threshold can be trusted. Such a result would identify a measurement limitation rather than a failure of threshold arithmetic.

For a three-axis implementation, preserve axis names through the complete processing chain. An accidental swap between channels can appear to be a new motor behaviour. Include a simple orientation check during commissioning, then document whether the monitoring decision uses one axis, separate axis thresholds or a defined combination. A change in this decision should produce a new configuration version and a renewed baseline check.

Table 5. Mounting record fields

Field	Record
Location	Housing reference and photograph identifier.
Attachment	Fastening method and condition before the run.
Orientation	Direction of X, Y and Z relative to the motor.
Cable route	Strain relief and routing description.
Remounting	Whether the sensor was moved since the last baseline.

i. Electrical integration and power arrangement

The system contains a sensing circuit and a motor power circuit. Their functions differ even when a common reference is needed for measurement. The sensor and controller require suitable low-voltage supplies, while the motor requires a supply appropriate to its own rating. A GPIO is a signal connection and should not be treated as a motor supply. The exact power arrangement must be recorded for the motor and development board actually used.

A connection plan should identify signals by function before assigning GPIO numbers. This avoids presenting a pin allocation for one ESP32 variant as universal. After checking the chosen board documentation, map each selected analog output to an available ADC channel. Record the connector labels, the signal names used in firmware and the reference ground. Review this mapping again after any rewiring.

Supply quality can affect an analog measurement. Motor operation can introduce disturbances through wiring, shared supply paths or nearby conductors. The integration procedure should compare motor-off and motor-on signals without changing the sensor attachment. If the controller resets or the signal changes abruptly when power is switched, investigate power integrity and grounding before evaluating anomaly detection.

Short, secure sensor connections reduce accidental disconnection and make the demonstration easier to inspect. Breadboards and temporary connectors are useful for a prototype, but they should be checked before repeated runs. A later enclosure or connector board can improve reliability once the wiring is settled. The project report should distinguish this planned packaging improvement from hardware that has actually been assembled.

Commissioning should proceed in stages: inspect connections without power, verify supplies, verify stationary sensor output and then operate the motor. At each stage record the observation needed to continue. This sequence localises integration problems and reduces the chance that an electrical problem is mislabelled as unusual vibration. It also provides an auditable explanation of how the final signal path was established.

Table 6. Functional connection plan

Signal	Connection rule
Sensor supply	Use the verified supply input of the installed module.
Sensor ground	Provide the common analog reference required by the controller.
X, Y or Z output	Connect selected axes to verified ADC-capable inputs.
USB interface	Use for programming and local serial observation.
Motor supply	Use the motor-rated supply arrangement recorded for the rig.

j. ADC configuration and calibration strategy

An ADC configuration is part of the instrument definition. It identifies the controller variant, selected channel, resolution, input range configuration and reading interface. The same raw sensor voltage can produce different digital results after a configuration change. A feature threshold expressed in ADC counts is therefore meaningful only with the acquisition configuration that generated it. Store that configuration with every test series.

Espressif distinguishes raw ADC conversion from calibrated voltage reporting and provides device-specific calibration support [3, 6, 7]. These resources describe the driver behaviour, rather than validating this project board. The implementation should use the interface appropriate to its chosen development environment and verify the resulting readings against a suitable reference where available. Do not assume that a raw count multiplied by a nominal voltage factor is a calibrated measurement.

Calibration has several layers. First establish that the ADC input is within range and not clipping. Then check the conversion from digital output to voltage. Finally, if acceleration units are needed, establish the sensor offset and sensitivity for each axis under the actual operating conditions. These layers should be documented separately. Correct ADC voltage conversion does not by itself establish the acceleration sensitivity of the sensor module.

For a relative vibration comparison, consistent ADC units can still be useful. The claim should then be restricted to changes in the same measurement arrangement. Moving from ADC counts to voltage or acceleration changes the units of both the feature and its limit. A configuration conversion should be verified with known values, and the old threshold should not be reused silently in the new units.

Calibration records should include the equipment or reference used, the applied conditions, the readings and the calculation. If a check cannot be completed, retain the uncalibrated unit label and state the corresponding interpretation limit. The correct response to incomplete calibration is a narrower measurement claim. Replacing the missing check with a nominal sensitivity would make the displayed precision look better without establishing accuracy.

Table 7. Calibration layers

Layer	Verification question
Input range	Can the expected sensor voltage be acquired without clipping?
ADC to voltage	Does the reported voltage agree with an independent reference?
Sensor offset	What output corresponds to the chosen static condition?
Sensor sensitivity	How does a known acceleration change the output?
Feature units	Do baseline, feature and threshold use the same units?

k. Sampling windows and response-time budgets

Sampling choices determine which changes the firmware can observe and how often it can update a decision. A nominal sample rate specifies the intended interval between readings. The window length specifies how many readings contribute to a feature. Their ratio gives the nominal window duration. This relationship should be checked before a demonstration so that the operator does not expect a warning faster than the acquisition process permits.

The following cases are arithmetic design examples, not firmware settings or measured response times. At 1000 samples per second, a window of 200 samples spans 0.2 seconds. Doubling that window to 400 samples doubles the span. At 2000 samples per second, the same 400 samples again span 0.2 seconds. These examples show why a window length alone is insufficient to describe a monitoring algorithm.

A warning delay also depends on the event position within the window. A change just after a new window starts may need most of the window to be collected before the feature can be calculated. Processing and output add further delay. If the decision requires several consecutive abnormal windows, that policy adds additional observation time. The test plan should therefore measure delay from the stimulus to the reported decision, not just the time spent in a calculation function.

Overlapping windows can update a feature more frequently than a completely new window, but the outputs then share samples. Frequent updates do not imply independent evidence. When evaluating false alarms, record the overlap and the unit of analysis. A sequence of warnings from heavily overlapping windows may represent one sustained event rather than many separate detected faults.

The design should select sampling and window settings after considering the retained signal band, sensor behaviour and controller workload. Verify the actual interval distribution under the same reporting mode used during a trial. A setting that works while serial output is disabled may behave differently when every reading is printed. Save the timing check alongside the selected configuration.

Table 8. Illustrative acquisition-window calculations

Sample rate	Samples	Window duration
1000 samples/s	200	0.200 s
1000 samples/s	400	0.400 s
2000 samples/s	400	0.200 s
2000 samples/s	800	0.400 s

l. Filtering and the retained vibration band

Filtering defines which parts of the signal contribute to the monitoring decision. A low-pass filter reduces higher-frequency content, while removing a window mean suppresses a steady component within that window. These operations address different problems. Mean removal does not protect the ADC from an out-of-band signal, because aliasing can occur during sampling before the digital mean is calculated. The filtering plan should therefore identify both the analog path and any later digital processing.

The retained band should follow the intended observation. A broad band may preserve more vibration detail while also admitting more noise and unrelated disturbances. A narrow band can make a particular comparison steadier but can exclude a relevant change. There is no universally correct filter for every motor. The report should state which response the prototype is intended to observe and confirm that the sensor module and acquisition settings can represent that response.

The installed module may already contain capacitors or other components that influence its response. Inspect those components or the module documentation rather than assuming the full bandwidth of the bare sensor. If the module is replaced, this response may change. That possibility is another reason to record the module identity and to repeat baseline measurements after hardware substitution.

Digital smoothing can improve readability of a feature trend, but excessive smoothing delays a warning and can hide short events. Keep the unsmoothed feature available during evaluation. If a smoothed value drives the decision, describe the smoothing rule and verify its response to an input change. A graph that looks cleaner is not automatically a more useful detector.

A practical validation sequence compares the raw signal, centred signal and calculated feature under the same labelled condition. Investigate unexpected high-frequency variation, clipping or slow drift before selecting a threshold. Change one processing choice at a time so that its effect can be identified. Freeze the filter configuration before evaluating detection and false alarms. Retuning the filter on the final test runs would weaken the independence of the evaluation.

Table 9. Processing operations and their roles

Operation	Main role	Limit
Analog band limitation	Reduce unwanted input content before sampling.	Requires hardware-aware design.
Window mean removal	Separate steady offset from dynamic amplitude.	Does not prevent aliasing.
Digital smoothing	Stabilise a displayed or decision feature.	Can delay or suppress events.
Clipping check	Identify measurements outside usable headroom.	Does not reconstruct lost signal.

m. Worked example of amplitude feature calculation

A short numerical example helps verify the arithmetic independently of the hardware. Consider four illustrative raw readings of 990, 1010, 990 and 1010 ADC counts. Their mean is 1000 counts. Subtracting this mean gives centred values of -10, 10, -10 and 10 counts. The mean square is 100 counts squared, so the centred RMS is 10 counts. The largest absolute centred value is also 10 counts.

Now consider an illustrative window of 980, 1020, 980 and 1020 counts. Its mean is still 1000, but the centred RMS and absolute peak are both 20 counts. The example separates a change in vibration amplitude from a change in the steady offset. Both windows have the same offset, so their different amplitude features have an immediate explanation. These numbers are mathematical inputs for verification and are not motor recordings.

A third example uses a constant reading of 1000 counts. Its uncentred RMS would be 1000 counts, while its centred RMS is zero. This shows why a steady accelerometer bias can dominate an incorrectly defined vibration metric. In a real stationary recording the feature is unlikely to be exactly zero because noise and movement remain. The arithmetic example isolates the definition rather than predicting the noise floor of the assembled system.

These cases can be replayed directly into a feature-calculation function. The expected values are known before the function runs. Add cases containing negative centred values, a single excursion and a constant offset added to every sample. The centred RMS should be unchanged by a constant offset when the calculation does not clip or overflow. A failure of this property identifies a processing error without requiring a motor trial.

Verification should also check numerical range. Squaring samples increases the magnitude of intermediate values, and summing many squares increases it again. Select accumulator types for the expected range and window length. Use a second independent calculation on a saved small window to check the firmware result. Store the expected and actual values with the software revision so that later changes can be checked against the same examples.

Table 10. Illustrative feature verification cases

Input counts	Mean	Centred RMS	Peak
990, 1010, 990, 1010	1000	10 counts	10 counts
980, 1020, 980, 1020	1000	20 counts	20 counts
1000, 1000, 1000, 1000	1000	0 counts	0 counts

n. Threshold selection and operating-condition control

A threshold expresses an acceptable region for a feature under stated conditions. It should be selected from repeated reference observations rather than from a single convenient screenshot. Start by collecting separate baseline runs after the motor reaches the chosen operating state. Examine both the typical values and the range of normal variation. Record any unusual windows and investigate their causes before deciding whether they represent acceptable baseline behaviour.

Several selection approaches are possible. A manually chosen limit can be compared with the observed baseline range. A percentile can describe an upper region of a baseline distribution. A mean plus a chosen multiple of the standard deviation can also be used as a candidate rule. These are selection methods, not guarantees of a particular false-alarm rate. Their behaviour depends on the data, dependence between windows and later operating conditions.

The limit should be assessed using runs that did not determine it. If it produces excessive warnings during independent baseline runs, investigate whether the measurement is unstable, the operating state changed or the candidate limit is unsuitable. Raising the limit without examining the cause may reduce warnings while also reducing sensitivity to the intended change. Evaluate both effects together.

An operating-condition label can prevent an inappropriate comparison. A motor at one speed or load may have a different normal vibration distribution from the same motor at another condition. The simplest demonstration can restrict monitoring to one controlled state. A broader system can maintain separate baselines, provided it can identify the current state reliably. An unknown state should not silently inherit a threshold selected for a different condition.

Every threshold revision should have a reason and a version. Record the data used to select it, the feature units, the acquisition settings and the resulting independent evaluation. Retain previous limits so that a warning in an older log can be reproduced. This history makes the detection logic reviewable and helps distinguish a real improvement from a change in the test conditions.

Table 11. Threshold configuration record

Field	Required meaning
Feature and axis	Exactly which quantity is compared.
Units and limit	Numerical value in the same units as the feature.
Operating state	Speed or load category to which the limit applies.
Selection evidence	Baseline runs used to choose the candidate.
Validation evidence	Separate runs used to assess the frozen limit.

o. Decision states and warning recovery

The output should distinguish an acceptable measurement from an unusable measurement. A small feature value can mean low vibration, but it can also arise from a disconnected or incorrectly acquired signal. The proposed state model therefore separates data validity from the normal or anomaly decision. The exact checks depend on the final hardware and firmware. Their purpose is to prevent missing or invalid data from appearing as evidence of a healthy motor.

An initialising state lasts while the controller prepares the ADC, resets buffers and establishes the required configuration. A monitoring state becomes available only after a complete usable window exists. The decision then reports normal or anomaly according to the configured rule. An invalid-reading state records that the current window cannot support the condition decision. The serial record should include the reason where the firmware can identify one.

A threshold comparison can alternate rapidly when the feature remains close to the boundary. Hysteresis addresses this by using one limit to enter a warning and a lower recovery limit to leave it. Persistence instead requires a condition to continue across a defined number of windows. These are different policies. Either can be evaluated as an enhancement, but both change warning timing and must be described in the configuration.

Recovery should be visible rather than implied by the disappearance of messages. Record the window that entered a warning and the window that cleared it. If a warning is latched for review, specify how an operator acknowledges it and whether measurement continues during that period. Acknowledgement and return to normal are separate events: an operator can acknowledge an active anomaly without making the feature return below its recovery limit.

The state model supports inspection rather than autonomous motor control. No shutdown action follows from the project presentation. Adding one would require a separate analysis of the actuator, failure behaviour and authority to stop the equipment. For the current scope, the useful output is a clear, timestamped condition indication accompanied by the feature values needed to understand it.

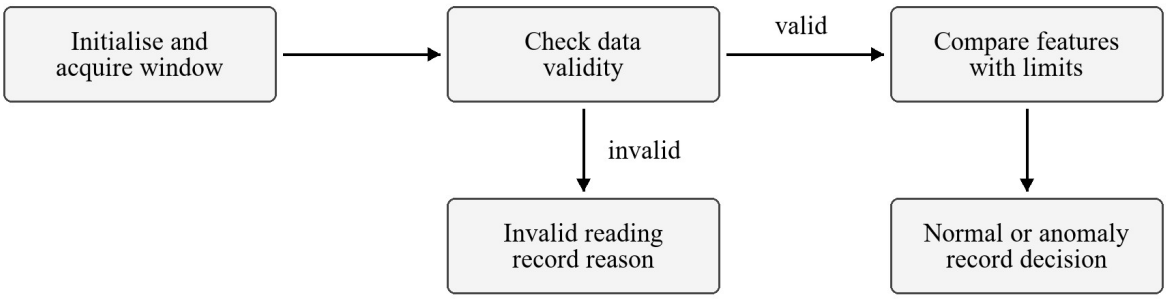


Figure 2. Proposed validity and decision-state flow

p. Firmware modules and information flow

A modular firmware design makes the monitoring logic easier to inspect and validate. The acquisition module owns channel selection and sample timing. A window manager records when a group of samples is complete. A feature module calculates the defined statistics, and a decision module compares valid features with the active configuration. The output module formats the record for serial observation. Each boundary has a small, explicit input and output.

The acquisition interface should return more than a bare sample when timing or status information is needed. For example, a sample record may include its channel, value and timestamp or sequence position. The exact representation can be compact, but it must preserve the information needed for validation. If three axes are read in sequence, record that the measurements were sequential rather than describing them as simultaneous acquisition.

The feature module should accept a completed window and return values with an associated validity result. It should not decide which maintenance action to take. Keeping the calculation separate allows known numerical windows to be used for software verification. The decision module can then be tested using prepared feature values without requiring the ADC or motor.

Configuration should have a single controlled definition. If a threshold is printed from one variable while the decision compares another, the log can misrepresent the rule. The same concern applies to sample rate, selected axes and units. Print a configuration header at the beginning of a recording and include a revision identifier in later records where practical.

Failure handling belongs at the interfaces. A missing window, invalid feature or unavailable output buffer should produce a deliberate response rather than an accidental default. Counters for missed samples or dropped output help identify whether the controller is keeping up. These functions are proposed implementation practices. Their presence in the final firmware must be confirmed by code review and targeted tests.

Table 12. Proposed firmware responsibilities

Module	Input	Output
Acquisition	ADC configuration and timing schedule.	Samples and acquisition status.
Window manager	Ordered samples.	Complete window and metadata.
Features	Valid sample window.	Centred RMS, peak and validity.
Decision	Features and active limits.	Condition state and reason.
Reporting	Configuration and decision record.	Serial line or logged record.

q. Timing buffers and resource estimates

The controller must collect the next samples while also calculating features and reporting earlier results. A simple sequential implementation can be sufficient for a demonstration if its timing is verified. The important question is whether processing or output delays disturb acquisition. Measure this under the intended operating mode rather than assuming that a short calculation is always harmless.

A full sample buffer makes waveform retention and independent recalculation straightforward. Its memory requirement follows from the number of channels, samples per channel and bytes stored per sample. As an arithmetic example, three channels with 256 values each and two bytes per value require 1536 bytes for the samples alone. Metadata, additional buffers and library use add to that amount. This is a design estimate, not a measurement of the prototype memory footprint.

A streaming feature calculation can reduce storage by accumulating a count, sum, sum of squares and extrema. However, centred variance calculations can suffer numerical cancellation when a large offset dominates small variation. A two-pass calculation on a retained window or a numerically stable online method can address that issue. The choice should be verified with known windows containing large offsets and small dynamic changes.

Double buffering separates the window being filled from the one being processed. This can improve continuity but introduces ownership rules: the acquisition task must not overwrite a buffer still being used. A single flag without a clear handover policy can create mixed windows that look plausible. Sequence numbers and overrun counters help expose such errors during integration testing.

Serial bandwidth also creates a resource budget. An illustrative record of 60 bytes sent 20 times per second requires 1200 payload bytes per second before communication overhead. Printing every raw sample can require much more. Decide whether a mode is intended for waveform capture or feature monitoring and evaluate its timing accordingly. Report an overrun honestly instead of treating delayed samples as if they arrived on schedule.

Table 13. Illustrative resource calculations

Item	Calculation	Result
Three-axis sample buffer	3 × 256 × 2 bytes	1536 bytes
Two such sample buffers	2 × 1536 bytes	3072 bytes
Feature record payload	60 bytes × 20 records/s	1200 bytes/s
Window span at 1000 samples/s	256 / 1000	0.256 s

r. Serial records and data dictionary

The Serial Monitor provides a readable view of messages, while the Serial Plotter displays numerical streams [8]. A useful demonstration should define which mode it is using. A plotting stream containing changing prose can be difficult to interpret, while a review log containing only numbers can lose the meaning of each field. Select a stable format for each purpose and retain a description of that format with the firmware.

A feature record should link the decision to the window that produced it. A sequence number establishes order even if a host timestamp is unavailable. A controller timestamp helps assess intervals, and a configuration identifier links the values to their settings. Features, limits and validity flags allow the warning to be checked later. The record should be self-consistent even when it is copied out of the serial window.

Choose units explicitly. A column called vibration is ambiguous because it could contain a raw sample, centred acceleration or RMS amplitude. A label such as rms_x_counts states both the statistic and its unit. If calibration later introduces millivolts or acceleration units, update the field name and configuration version. Do not combine records in different units in the same analysis without an explicit conversion.

Plotting often benefits from fewer simultaneous values. During waveform inspection, plot the selected raw or centred signal. During decision inspection, plot a feature with its threshold and retain the textual state in the associated log. Keep a stable numerical order. The exact formatting syntax should be verified against the Arduino IDE version used for the demonstration.

The computer display is an observation interface rather than a durable archive. Save the numerical output to a file when evidence is needed, and confirm that the file contains the complete run. A screenshot can show the user interface but cannot replace the underlying samples for recalculation. Use both where they answer different review questions.

Table 14. Proposed feature-record fields

Field	Meaning
run_id and config_id	Link the record to test conditions and firmware settings.
window_id and time	Preserve order and timing context.
rms_x_counts	Centred RMS on the selected axis in ADC counts.
peak_x_counts	Absolute centred peak on that axis.
limit and state	Expose the comparison and resulting indication.
valid and reason	Distinguish unusable data from a normal decision.

s. Data-quality checks and exceptional conditions

Data quality must be evaluated before condition interpretation. An input that stays at one extreme, a missing channel or an incomplete window can distort the feature. Some faults can be identified by range checks or acquisition status, while others require an independent observation. The firmware should apply the checks it can support and avoid presenting those checks as complete coverage of every sensor failure.

Clipping occurs when the observed signal reaches a measurement limit and further change is lost. A clipped waveform can make the peak appear constant and can alter RMS. Record the number or fraction of samples near configured limits, together with the criterion used to flag them. The detection threshold should not be tuned to compensate for clipping. Correct the acquisition range or reduce the input within the intended measurement arrangement.

A constant signal needs careful interpretation. A stationary sensor can be quiet, but a broken connection may also appear steady. Variation alone is therefore not a universal connection test. Combine the information available from wiring checks, a commissioning response test and the motor operating state. If validity remains uncertain, report that uncertainty through an invalid or unverified state rather than assuming normal operation.

Timing quality can be checked independently of amplitude. A complete count of samples does not guarantee evenly spaced acquisition. Record missed deadlines, unexpected intervals or window overruns where possible. A later feature-analysis script should be able to exclude or separately inspect windows that fail the timing requirement. This prevents a communication delay from being mistaken for a mechanical change.

Exceptional-condition tests should be planned before the demonstration. Include startup, reset, loss of the host serial connection and a controlled sensor-connection check performed with the rig in a suitable state. Record how the monitor resumes and whether its baseline and counters remain interpretable. Recovery behaviour is part of reliability because an unattended-looking normal display after a reset can otherwise conceal a gap in observation.

Table 15. Quality checks and interpretation

Observation	Possible issue	Required response
Persistent range-limit readings	Clipping or incorrect input configuration.	Flag and inspect the acquisition path.
Incomplete sample window	Missed acquisition or buffer problem.	Exclude the window from condition scoring.
Unexpected sample intervals	Scheduling or output interference.	Record timing failure and investigate.
Unexplained constant signal	Quiet input or connection problem.	Verify the sensor response independently.
Reset during a run	Power or software interruption.	Mark the discontinuity and restart the run record.

3. EXPERIMENTS

a. Setup and test procedure

The experimental objective is to check whether the complete sensing and processing chain produces repeatable readings and distinguishes the chosen baseline from controlled vibration changes. The following procedure defines proposed validation work. It does not imply that the trials have already been completed. Each run should record the motor condition, mounting arrangement, selected axis, firmware version, sample rate, window size and thresholds.

First verify the sensor supply, common ground and ADC input connections. With the motor off, observe the sensor readings and check that they respond consistently to changes in orientation without clipping. Attach the sensor at the chosen motor location and secure its wires. Start the motor under a stable baseline condition and collect multiple windows after the signal settles. Save the numerical output for later comparison, together with the time and test label.

Next repeat the baseline run to assess normal variation. Introduce only controlled, supervised changes suitable for the laboratory rig, or use a safe external vibration stimulus to check the signal path. Keep the sensor attachment unchanged during comparative runs. A stimulus test checks sensitivity to a changed signal but does not establish a specific motor defect. Avoid uncontrolled damage or contact with rotating parts.

The threshold-selection data and evaluation data should be kept separate. Otherwise a limit tuned on a particular recording may appear more effective than it is on a new run. Repeat each relevant condition and record any warnings with their timestamps. Include startup and shutdown as separate operating states, since their transients may differ substantially from steady operation. Decisions about suppressing these transients should be documented.

Table 16. Proposed experimental test matrix

Test condition	Purpose	Record or check
Motor off	Check sensor bias and background variation.	Raw signal, clipping and centred amplitude.
Stable baseline	Establish normal operating variation.	RMS and peak across repeated windows.
Repeated baseline	Check repeatability and false alarms.	Feature spread and warning count.
Controlled vibration change	Check response to an increased or altered input.	Feature changes and exceedance timestamps.
Startup and shutdown	Identify transient behaviour.	Transient warnings and recovery behaviour.
Changed load or speed, if available	Assess dependence on operating condition.	Separate condition labels and baseline suitability.
Sensor or connection check	Assess handling of an invalid input.	Invalid-reading output where supported.

b. Evaluation measures

Evaluation should distinguish a waveform illustration from a measured recording. Useful evidence includes saved ADC or voltage samples, feature logs, actual serial screenshots and a labelled test log. A complete record links each output to its known test condition. Measuring only whether a line appears in the Serial Plotter is insufficient to estimate detection performance.

Table 17. Evaluation measures and recording method

Measure	Definition or recording method
RMS vibration	Centred root mean square feature per window, with units and selected axis.
Peak amplitude	Largest absolute centred sample in the window, using the same units.
Detection rate	Detected labelled abnormal trials divided by all labelled abnormal trials; state the decision unit and count.
False-alarm rate	Normal trials or windows with a warning divided by all normal trials or windows, using one consistent decision unit.
Response time	Elapsed time from a timestamped stimulus or condition change to the first valid warning.
Repeatability	Variation in features across comparable baseline runs with the same mounting and operating state.

Response time includes the acquisition window, processing and reporting delay. A longer window may make the feature steadier while delaying an indication. Persistence checks can add further delay. Detection and false-alarm rates require independently labelled conditions and adequate repetitions. Without labelled reference conditions, a threshold crossing can be reported as an observed event, but it cannot be converted into a meaningful accuracy percentage.

The test record should preserve raw or minimally processed readings where possible. This allows unexpected warnings to be investigated rather than attributed automatically to a motor fault. If calibration is incomplete, comparisons can still be made in consistent ADC units, provided the report avoids presenting them as calibrated acceleration. Future trials can add a reference vibration instrument to assess measurement agreement.

c. Commissioning and calibration protocol

Commissioning establishes that the assembled system measures the intended signal before a performance experiment begins. Start with an identified controller, sensor module and firmware version. Review the functional wiring plan and inspect the attachment. Record the power arrangement and the selected axes. These records establish what was tested and should be completed during the work.

With the motor off, verify the relevant supply and input conditions using suitable equipment. Confirm that the sensor output changes consistently during a controlled orientation check. This establishes basic response and channel identity, although it does not establish dynamic vibration calibration. Record whether the check used raw counts or calibrated voltage and preserve the observations for every axis used.

If an independent voltage reference is available, compare controller readings with that reference at appropriate points within the intended range. Stay within the permitted electrical limits. Record the reference value, reported value and conditions rather than writing only a pass label. A later reviewer can then judge whether the difference is acceptable for the purpose of the project.

Acceleration calibration requires a separate reference and method. A static orientation check can help examine offset and response to gravity, but it does not characterise the complete dynamic response of the mounting and sensor system. If dynamic calibration equipment is unavailable, retain relative amplitude units and limit the claim accordingly. Repeatability and threshold behaviour can still be evaluated in those consistent units.

After the electrical checks, secure the sensor and acquire motor-off and stable motor-on records. Inspect clipping, baseline variation and timing. Define acceptance criteria before using these records for evaluation. Where a criterion is qualitative, identify the observation and reviewer decision explicitly. Commissioning is complete when the evidence supports proceeding, not merely when a waveform appears on the screen.

Table 18. Commissioning evidence checklist

Stage	Evidence to retain
Identity and wiring	Board identity, firmware revision and connection record.
Stationary response	Axis-response observations with units.
Voltage comparison	Reference and reported values, where available.
Mounting and operation	Attachment photograph and operating condition.
Acquisition readiness	Range, timing and baseline-quality observations.

d. Controlled experimental design

An experiment should isolate the change being investigated. If speed, mounting and sensor orientation all change between recordings, a different feature value cannot be attributed to one cause. Define the independent condition, the quantities held constant and the outputs recorded. Use the same acquisition and processing configuration unless the purpose is specifically to compare configurations.

Repeat baseline runs rather than treating one run as a permanent reference. Collect records from more than one start of the motor and preserve run boundaries. Repeated starts can reveal variation hidden in a long uninterrupted recording. If the motor warms or the load changes over time, note that progression and consider its effect when comparing early and late runs.

Comparative tests can use a supervised change suitable for the laboratory setup or a controlled external vibration stimulus. Label the experiment according to what actually changed. An external stimulus is a changed-input test, not proof that an internal motor defect was detected. Do not create an unsafe mechanical condition to obtain a larger waveform. The evidence should support the narrow claim being tested.

Run order can influence interpretation. If every baseline trial occurs first and every changed-condition trial occurs later, temperature, supply conditions or sensor movement may align with the labels. Where practical, alternate comparable runs and inspect the sensor between them. Record the order so that an apparent detection improvement can be checked against changes over time.

Record exclusions as part of the experiment. If a cable came loose, the controller reset or the intended motor condition was not maintained, mark the run and explain its treatment. Do not remove inconvenient records without a documented rule. Excluded data can reveal integration weaknesses even when it is unsuitable for calculating performance under valid conditions.

Table 19. Variables in a comparative trial

Role	Examples
Independent condition	Baseline or a specified controlled change.
Controlled conditions	Mounting, axis, supply and firmware settings.
Recorded outputs	Samples, features, status and timing.
Confounding influences	Speed variation, warming and bench disturbance.
Exclusion conditions	Reset, invalid acquisition or loss of the intended state.

e. Labels and separation of evaluation data

A detection metric requires a reference label independent of the detector output. Calling a window abnormal because the monitor raised a warning makes the evaluation circular. Use the documented experimental condition and an independent record of when it applied. If the true condition is uncertain, mark it uncertain instead of forcing it into a normal or abnormal category.

Choose the decision unit before counting outcomes. A window-based evaluation classifies each valid sample window. A trial-based evaluation asks whether an event was detected during a defined run. These units answer different questions. A long abnormal trial can produce many windows while a trial-based count treats it as one event. State the unit and retain the observation counts for each measure.

Keep threshold-selection runs separate from evaluation runs. If the same data both selects and tests the limit, the result describes how well the rule fits those recordings. Separation is particularly important when neighbouring windows overlap or come from one uninterrupted session. Randomly splitting individual windows can leave almost identical samples on both sides of the split.

Transition periods need a policy. A stimulus may begin partway through a window, producing a mixture of baseline and changed behaviour. Define whether such windows are used for latency analysis, assigned a transition label or excluded from steady-state counts. Preserve timestamps so that another reviewer can apply the same rule. Do not shift the event time to make the response appear faster.

When several operating states are evaluated, retain their separate labels as well as the overall reference class. A combined score can conceal a poor result at one speed or load. Report counts by condition where the repetitions support comparison. With small samples, describe the observed counts without presenting them as a universal estimate of industrial performance.

Table 20. Proposed label structure

Field	Purpose
run_id	Identify the independent recording session.
condition	Describe the actual motor or stimulus condition.
event_start and event_end	Link the controlled change to time.
reference_class	Normal, changed, transition or uncertain.
validity	Separate acquisition failure from condition class.
data_role	Commissioning, selection or final evaluation.

f. Robustness and negative-condition tests

Test conditions that might produce an inappropriate warning as well as conditions intended to produce a warning. A controlled baseline comparison alone does not reveal behaviour during startup, remounting or a disturbed supply. Negative-condition tests investigate whether these influences are recognised, recorded or confused with a change in motor condition.

Begin with normal operational variation expected in the demonstration. Repeat motor starts, stable runs and shutdowns with the same attachment. Observe whether warnings cluster around transitions. If monitoring is intended only during steady operation, define the entry condition for that mode. A manual declaration of steady state should be recorded rather than assumed to be an automatic capability.

Examine the acquisition path in a controlled state. Check a planned sensor disconnection or reconnection only when suitable for the rig and supply arrangement. Verify whether the firmware reports invalid input, a normal-looking constant value or another state. Discover the actual behaviour rather than claiming a connection-check mechanism that the firmware does not implement.

Describe environmental influences precisely. An external stimulus can reveal sensitivity to bench disturbance, while a changed cable route can reveal coupling or attachment problems. Change one influence at a time and restore the baseline arrangement afterward. If the restored baseline differs, investigate whether the intervention moved the sensor or altered its support.

Robustness findings should feed back into the design. A repeated reset suggests a power or software problem. Warnings during valid baseline operation suggest an unsuitable limit or insufficient operating-state control. A normal indication during invalid acquisition suggests a reporting problem. Classify each issue by its cause and verification evidence so that a later correction can be retested.

Table 21. Proposed robustness cases

Case	Question
Repeated starts	Are transitions identified consistently?
Baseline repeats	How often does an unwarranted warning occur?
Connection check	Does unusable input appear as normal?
Host interruption	Does acquisition continue and is the gap visible?
Bench disturbance	Can the environment mimic the chosen anomaly?
Power-cycle recovery	Is the new run clearly identified?

g. Response-time measurement procedure

Measure response time between a known condition change and a defined reported event. Select both endpoints before testing. The start may be a timestamped external stimulus or a recorded transition in a controlled test. The endpoint may be the first anomaly decision generated by the firmware. A host display update is a different endpoint because it includes communication and display delay.

Use a common time basis where possible. If a host records the stimulus and a controller clock records the decision, establish their relationship before subtracting timestamps. Alternatively, capture both endpoints with one independent measurement system. A screen recording can support a demonstration, but its timing resolution and endpoint ambiguity limit how short a delay it can establish.

Repeat the event at different positions within the acquisition window. The delay can depend on whether the change occurs near the beginning or end. Report the observed range as well as a typical value when sufficient trials exist. Keep persistence and smoothing settings fixed. Removing persistence to obtain a faster warning changes the policy, rather than improving timing under the same rule.

Separate detection delay from acquisition failure. A window that misses deadlines may produce a late feature even when arithmetic executes quickly. Record the sample interval, window completion time, decision time and output time where supported. These intermediate observations identify whether delay arises in measurement, processing or communication.

Also measure recovery. Record how long the indication remains active after the feature returns to its acceptable region, using the configured rule. A warning may be intentionally latched. Describe this policy so its duration is not mistaken for continued vibration. Retain the timestamps and observations required to reproduce both entry and recovery calculations.

Table 22. Timing endpoints to record

Endpoint	Meaning
Condition change	Independent reference for event start.
Window completion	Relevant sample group becomes available.
Decision generation	Firmware applies the configured rule.
Record transmission	Status leaves the controller.
Display observation	Host visibly reflects the indication.
Recovery decision	Configured recovery condition is met.

h. Integration tests and requirements traceability

Traceability connects each requirement to a specific observation. A successful demonstration of one function is not evidence for every function. A waveform in the Serial Plotter shows that some data reaches the computer, but it does not establish correct RMS arithmetic, valid timing or warning recovery. Each of these needs a targeted test.

Calculation tests use prepared numerical windows with known answers. Acquisition tests inspect channel mapping, input response and timing. Decision tests apply known features around the threshold. System tests combine assembled hardware with labelled conditions. Keeping these levels separate makes a failure easier to localise and helps the team explain precisely which part of the design was verified.

For each test, record the precondition, action, expected behaviour and observation. Expected behaviour follows the requirement and configuration. At the exact threshold boundary, the comparison operator determines whether a warning should appear. Verify the equal-to case explicitly instead of relying only on obviously high and low values. Also check how the decision responds when the input validity flag is false.

Support every pass label with a record identifier, such as a calculation sheet, serial log or acquisition trace. Mark an unperformed test as not executed with a reason. It should not be silently counted as passed. This distinction is especially important for invalid-input handling because available checks depend on hardware details that the generic design cannot guarantee.

After a correction, rerun the test that exposed the issue and related integration checks. Changing output frequency can affect acquisition timing even when the change seems confined to the display. Record the firmware revision used for each outcome. The completed test matrix then explains what the final prototype demonstrated and what still needs validation.

Table 23. Proposed integration tests

ID	Target	Expected evidence
IT01	Channel mapping	Known response appears on the intended channel.
IT02	Arithmetic	Prepared windows match independent calculations.
IT03	Threshold boundary	Below, equal and above cases follow the rule.
IT04	Timing	Intervals meet the acquisition requirement.
IT05	Recovery	Entry and clearing follow the active policy.
IT06	Invalid input	Observed status matches documented capability.
IT07	Configuration	Logged settings match the decision settings.

i. Reproducibility and evidence management

A reproducible report allows another team to identify the setup, repeat the procedure and understand the resulting records. It does not require every sample to be identical. It requires control of meaningful conditions and a record of remaining variation. Preserve both technical settings and physical setup information so that a comparison has a defined basis.

Create a separate record for every independent run. Its identifier should link numerical data, condition notes, firmware revision and mounting photograph. Use file names that remain understandable after files move to another folder. Keep original data unchanged and analyse a copy. This preserves the ability to trace filtering and exclusions back to the acquisition record.

Record the development-board variant and software environment. Library changes can alter driver behaviour or available configuration options. Retain the version used for evaluated runs with the source. If the firmware is rebuilt later, compare settings and known-input calculations before assuming that previous performance evidence still applies. A matching project title is not sufficient configuration identification.

Analysis records should show which runs were included, which were excluded and how measures were calculated. Keep the decision unit consistent. If two analysis versions produce different metrics, retained labels and calculation steps should make the difference explainable. Spreadsheet formulas should remain inspectable rather than being replaced by a final percentage without its denominator.

The evidence set should support review of individual claims. Measurement claims need samples and calibration context. Detection claims need independent labels and warning records. Timing claims need endpoints and timestamps. Maintenance-value claims need operational follow-up beyond the prototype. Matching each claim to the evidence required prevents a laboratory observation from becoming an unsupported general conclusion.

Table 24. Suggested run evidence package

Artifact	Contents
Run summary	Condition, notes and start/end identifiers.
Configuration	Board, firmware, units, channels and limits.
Samples	Original values with order or timing.
Feature log	Statistics, validity and decisions.
Setup image	Attachment position and orientation.
Analysis record	Inclusions, calculations and review notes.

j. Troubleshooting and error isolation

Troubleshooting should follow the signal path rather than begin by changing the threshold. Establish that the intended sensor output reaches the selected ADC input, then inspect sample validity and timing. Only after these checks should feature arithmetic and decisions be investigated. This avoids compensating for a wiring problem with a setting that changes only its visible symptom.

If the signal appears constant, inspect units, channel mapping and sensor response during a controlled stationary check. A constant plot can also result from display scale or repeated reporting of one stored value. Compare the raw record with the displayed field. If acquisition changes while the displayed field does not, investigate the processing and output stages.

If amplitude becomes unusually large, check clipping, mounting and operating state before interpreting it as a motor fault. Confirm that mean removal uses the correct window and that arithmetic intermediates do not overflow. Recalculate a retained window independently. Correct a reproducible calculation error in the feature module and verify it using prepared numerical cases.

If warnings are late or inconsistent, inspect window duration, persistence and communication load. Measure the interval between acquisition and reporting. Reduce output only as a controlled diagnostic change, then repeat timing checks in the final reporting mode. Improvement with a quiet interface may identify an output bottleneck, but it does not complete the final configuration test.

End every resolved issue with a documented cause and repeat test. An improvement after reseating a connector is evidence, but may not completely identify the root cause. Record what changed and whether the symptom returned. This history helps a later team distinguish an integration weakness from a new issue and prevents repeated threshold changes from obscuring measurement problems.

Table 25. Troubleshooting paths

Symptom	First checks
Constant or missing values	Mapping, connection, units and live data path.
Maximum readings	Range, clipping and electrical configuration.
Baseline shift	Attachment, axis, speed/load and supplies.
Incorrect feature	Independent arithmetic and accumulator range.
Late warning	Window length, persistence and output load.
Reset or record loss	Power integrity and recovery behaviour.

4. RESULTS

a. System outputs and waveform interpretation

The monitoring approach provides two forms of output: a numerical view of vibration and a rule-based condition indication. Raw or centred samples can show the signal shape in the Serial Plotter. Window features and status messages can show the decision in the Serial Monitor. A useful output record associates the readings with a timestamp or window number so the signal and warning can be compared directly.

Figure 3 reproduces the waveform illustration used in the project presentation. The left trace represents normal behaviour and the right trace represents abnormal behaviour. The drawing communicates a difference in signal amplitude and variation. It has no calibrated axes or recorded test conditions, so it supports explanation of the monitoring concept rather than a numerical comparison of motor performance [1].

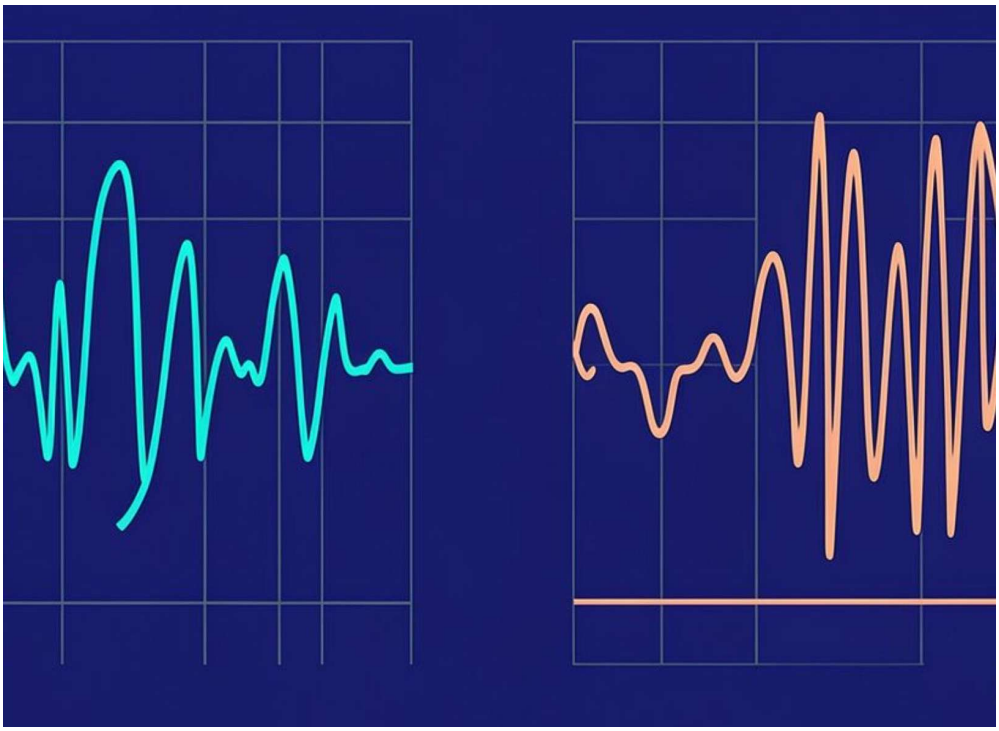


Figure 3. Illustrative normal and abnormal vibration waveforms
Left: normal illustration. Right: abnormal illustration. These are not measured results.

For an actual test, an anomaly output would mean that the selected feature exceeded its configured limit for the applicable decision rule. Interpretation requires the operating state and the measured feature value. An increased RMS or peak may justify checking the equipment, its mounting and the sensor connection. It cannot establish whether the cause is a bearing problem, imbalance, a load change or an unrelated disturbance without additional evidence.

b. Scope and limitations

Table 26. Project scope and interpretation

Item	Scope or interpretation
Embedded platform	ESP32 acquisition and embedded C/C++ development using the Arduino IDE.
Sensing method	ADXL335 analog vibration measurement and ADC sampling.

Item	Scope or interpretation
Decision method	Baseline or threshold comparison with an explicit rule.
Observation interface	Serial Monitor and Serial Plotter for the local demonstration.
Performance validation	Requires labelled recordings, repeat trials and measured timing.
Wireless services and machine learning	Future extensions requiring implementation and separate evaluation.

The expected benefit is earlier awareness of a vibration change using accessible embedded hardware. Potential applications include motor-driven pumps, conveyors, HVAC equipment and laboratory demonstrations. These applications describe where the approach may be useful. They do not establish industrial reliability or a verified reduction in downtime. Deployment decisions should follow testing under the relevant motor, load and environmental conditions.

Threshold detection is sensitive to the reference condition. A single limit may be unsuitable when motor speed, loading or mounting changes. Sensor bandwidth and the ADC configuration also limit which vibration components are represented accurately. Mechanical coupling, electrical noise and aliasing can alter the signal. Simple amplitude features may indicate that behaviour changed while providing limited information about the specific cause.

No numerical detection accuracy, false-alarm rate, response time or cost saving is claimed here. These quantities require a completed test record and a stated calculation method. A defensible results table should include measured values, units, trial counts and condition labels. The present system description establishes how those results should be obtained and interpreted, while the illustration explains the intended difference between baseline and abnormal behaviour.

c. Illustrative detection-score calculation

Performance measures should be calculated from explicit counts. Consider a teaching example containing 10 labelled changed-condition trials and 30 labelled normal trials. Suppose the rule warns on 8 changed trials and misses 2. Suppose it also warns on 3 normal trials and remains normal on 27. These counts are deliberately hypothetical. They explain the evaluation method and do not describe tests of the assembled ESP32 prototype.

The detection rate, also called sensitivity in a binary evaluation, is 8 divided by 10, or 80 percent. The false-alarm rate on normal trials is 3 divided by 30, or 10 percent. Precision is 8 divided by all 11 warning trials, approximately 72.7 percent. Accuracy is the 35 correctly classified trials divided by 40, or 87.5 percent. Reporting the counts beside the percentages makes the denominators visible.

These measures answer different questions. Detection rate describes how often the labelled changed trials produced a warning. Precision describes how often a warning corresponded to a changed trial in this particular collection. Its value depends on the mix of normal and changed trials. A different operating population can therefore produce a different proportion of useful warnings even when the detector behaviour on each class remains similar.

The decision unit must stay consistent throughout the calculation. If the denominator counts trials, the numerator must also count trials. Replacing the 40 trials with hundreds of overlapping windows would change the evaluation and its dependence structure. A long recording should not create a misleading impression of many independent experiments simply because the firmware generates frequent updates.

A real results table should replace this example only after independently labelled trials are completed. Retain the frozen configuration, all valid trial outcomes and the handling of uncertain labels. Report missed detections and false alarms as well as successes. If there are too few trials for a stable estimate, present the observed counts and their limited coverage rather than treating a precise percentage as proof of general reliability.

Table 27. Hypothetical scoring example

Outcome or metric	Illustrative value
True positives / false negatives	8 / 2 changed-condition trials.
False positives / true negatives	3 / 27 normal trials.
Detection rate	8 / 10 = 80%.
False-alarm rate	3 / 30 = 10%.
Precision	8 / 11 = approximately 72.7%.
Accuracy	35 / 40 = 87.5%.

d. Interpreting trends and responding to warnings

A warning becomes useful when an operator can relate it to equipment condition and choose an appropriate next step. The first response is to inspect the measurement context. Confirm the active configuration, sensor attachment and operating state. A large feature produced during a changed load should not automatically be interpreted using a baseline selected at another load. Preserve the warning record before making adjustments.

A trend provides context that one window cannot supply. Repeated elevated features under comparable conditions can justify closer examination, while an isolated peak may require investigation of a brief event or measurement disturbance. The distinction should be based on retained records and a defined observation period. It should not depend on whether the operator happens to notice a fluctuation on the screen.

An invalid-reading indication requires restoration of the measurement path. It should not be cleared by increasing the vibration limit. Check the reported reason, wiring and acquisition behaviour. If a reset or interruption occurred, mark the gap and establish a new run boundary. Continuing a graph without identifying the discontinuity can create a false appearance of continuous observation.

Maintenance decisions remain with the responsible operator or laboratory supervisor. The prototype can indicate that the measured behaviour changed, but it does not establish a repair instruction. A later inspection may identify a loose support, a changed operating condition or a mechanical issue. Recording that follow-up is valuable because it helps determine whether warnings correspond to actionable events.

Longer-term evaluation should connect warnings with inspection outcomes and subsequent operation. This is the evidence needed to assess practical maintenance value. A demonstration of threshold crossing supports the detection concept. A claim of avoided downtime or reduced maintenance cost would require additional operational records, costs and a comparison method. The present design creates an information path through which such evidence could later be collected.

Table 28. Proposed operator interpretation guide

Indication	Next step
Normal with valid data	Continue observation under the stated operating condition.
Isolated warning	Retain the record and check for a transient or disturbance.
Persistent warning	Review comparable baseline data and arrange appropriate inspection.
Invalid reading	Restore and verify the measurement path.
Condition changed	Check whether a different baseline or test label is required.

e. Limitations and staged development

The principal limitation is the gap between amplitude-change detection and fault diagnosis. A threshold can identify a selected kind of departure, but several physical causes can produce similar amplitude changes. A low feature also cannot rule out a defect that lies outside the sensor bandwidth, observation direction or chosen feature. The final interpretation should remain tied to the measurement that was actually validated.

Validity also depends on the range of tested operating conditions. A threshold evaluated on one laboratory motor should not be described as universal. Differences in support structure, motor size, load and mounting can change the observed distribution. Extending the application requires new baselines and evaluation, even if the embedded software can be reused with minor changes.

The next development stage should strengthen measurement evidence. Confirm the channel mapping, useful frequency band, timing and feature units. Then complete repeated baseline and changed-condition trials with retained raw data. Improvements in filtering or persistence should be compared using the same labelled evaluation procedure. Change control matters because simultaneous adjustments make it difficult to identify which change produced a different result.

Remote monitoring can follow a validated local loop. A dashboard should display timestamp, units and data validity as well as the latest amplitude. A disconnected device should not leave an old normal value looking current. Notifications should represent a defined event policy and should be tested for repeated or missing messages. These are future interface requirements rather than implemented features of the project described in the presentation.

Machine-learning extensions would require a suitable dataset and an independent validation design. Additional model complexity cannot recover vibration components that the hardware never measured. Establish consistent sensing and reliable labels first, then compare a trained method with the threshold baseline on separate runs. This staged approach preserves an interpretable reference and makes each claimed improvement testable.

Table 29. Development stages and evidence gates

Stage	Evidence required before extension
Measurement	Verified range, timing, units and repeatability.
Local detection	Independent labelled trials and warning records.
Condition coverage	Separate evaluation across relevant states.
Remote interface	Freshness, connection and notification checks.
Advanced analytics	Suitable labelled data and independent comparison.

5. Conclusion and Future Work

ESP32-based vibration monitoring provides a practical foundation for observing motor condition with accessible hardware. The project combines an ADXL335 accelerometer, analog acquisition and an embedded processing loop. The approach connects a physical vibration signal to amplitude features and an explainable threshold decision. Serial observation helps relate the sensor input to the reported status during development and demonstration.

The principal value of the method is a clear indication that the measured vibration has departed from a selected reference condition. Such a warning can support inspection and maintenance planning. Reliable use depends on repeatable sensor attachment, suitable signal acquisition and thresholds appropriate to the motor operating state. Establishing detection performance requires independent, labelled tests rather than waveform illustrations alone.

Further development should first improve the measurement and validation process. This includes recording sampling behaviour, checking sensor and ADC calibration, selecting filtering for the vibration band of interest and collecting repeated runs under known conditions. Condition-specific baselines can help distinguish changes in normal speed or load from changes that justify inspection. Longer observation periods can show whether amplitude trends are useful for maintenance decisions.

Additional sensors, such as temperature sensors, can provide complementary information about motor operation. A wireless interface can later transmit measurements to a dashboard or send a notification when a warning persists. These functions should follow a working local measurement loop and include evaluation of communication delays and interruptions. Frequency-domain features and trained machine-learning models are further options once a suitable labelled dataset exists.

The next stage is therefore to validate the complete prototype against a documented test procedure and retain the resulting signal logs. That evidence will determine which vibration changes the system can detect consistently and how quickly it reports them. With these checks, the project can develop from an understandable embedded monitoring demonstration into a better-supported tool for motor maintenance decisions.

References

[1] Mohammad Reyhan, Syed Faizan and Amaan Pasha Shaikh, ESP32-Based Predictive Motor Maintenance, Embedded Systems Project Review, project presentation, 10 slides.

[2] Analog Devices, ADXL335 Small Low Power 3 Axis ±3 g Accelerometer, Rev. B data sheet. https://www.analog.com/media/en/technical-documentation/data-sheets/adxl335.pdf (accessed 29 October 2026).

[3] Espressif Systems, Arduino ESP32 ADC API documentation. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html (accessed 29 October 2026).

[4] Pete Sopcik and Dara O'Sullivan, How Sensor Performance Enables Condition-Based Monitoring Solutions, Analog Devices. https://www.analog.com/en/resources/analog-dialogue/articles/how-sensor-performance-enables-condition-based-monitoring-solutions.html (accessed 29 October 2026).

[5] NI, Sensors for Condition Monitoring. https://www.ni.com/en/shop/data-acquisition/sensor-fundamentals/sensors-for-condition-monitoring.html (accessed 29 October 2026).

[6] Espressif Systems, Analog to Digital Converter Oneshot Mode Driver, ESP32 ESP-IDF Programming Guide. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc/adc_oneshot.html (accessed 29 October 2026).

[7] Espressif Systems, Analog to Digital Converter Calibration Driver, ESP32 ESP-IDF Programming Guide. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc/adc_calibration.html (accessed 29 October 2026).

[8] Arduino, Using the Serial Plotter Tool, Arduino IDE 2 documentation. https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-serial-plotter (accessed 29 October 2026).

APPENDIX A PROCESSING PSEUDOCODE

The following pseudocode specifies the information flow of an illustrative implementation. It is not a verified copy of the uploaded project firmware. The real channel mapping, window length, sampling interval and limits must be selected for the installed hardware and documented in a configuration record. No network or motor-control action is assumed.

A complete window is checked before its features enter the condition rule. The algorithm keeps invalid acquisition separate from a normal result. The optional persistence or recovery policy can be implemented inside the decision function, provided its behaviour is recorded and tested. Acquisition timing should be verified under the final output mode.

```
load and validate acquisition configuration
initialise ADC inputs and serial interface
print run identifier and configuration header
repeat while monitoring is enabled:
    reset window counters and accumulators
    for each scheduled sample position:
        wait for the next acquisition instant
        acquire the selected channel values
        record ordering and acquisition status
        store sample or update verified accumulators
    if the window is incomplete or invalid:
        set status to INVALID READING
        attach the available reason and counters
    otherwise:
        estimate the mean for each selected axis
        calculate centred RMS and absolute peak
        verify numerical and range validity
        compare valid features with active limits
        apply the configured recovery policy
        set status to NORMAL or ANOMALY
    report window identifier, features and status
    preserve any timing or output overrun counters
    continue with the next window
```

Review the deployed source against this sequence. Record any implemented changes in ordering, validity checks or decision policy before comparing test results.

Appendix B Demonstration and operating procedure

Prepare the demonstration with the motor secured and the sensor attachment documented. Identify the controller, selected axes and firmware configuration. Confirm that the intended local serial interface is available and that the power arrangement matches the recorded setup. The operator should be able to explain the signal path before the motor begins running.

Begin with a stationary check. Observe that the sensor inputs are within the intended range and that the chosen channels respond as expected. Record any quality warnings. Start a new run record and retain the configuration header. Then operate the motor under the selected baseline condition and allow the state to settle before collecting the baseline comparison.

During baseline observation, show both a numerical feature and the corresponding decision. Explain the unit of the feature and the origin of the configured limit. Use the Serial Plotter when the aim is to explain waveform behaviour, and the log when the aim is to explain window decisions. Do not change the threshold during a scored trial without ending the run and recording the new configuration.

For a controlled comparison, state the intended change and the evidence that marks its start. Keep the sensor mounting and other controlled conditions unchanged. Observe whether the feature changes and whether the configured rule reports an exceedance. Distinguish an actual saved recording from the illustrative normal and abnormal image included in the report.

After the comparison, restore the baseline condition and observe recovery. End the run, save the data and record any unexpected behaviour. Review the evidence before making a performance claim. A successful demonstration shows a complete sensing and decision chain; a numerical accuracy claim additionally requires the labelled repetitions and counting procedure described in the experiments section.

Table 30. Demonstration review points

Point	Operator explanation
Signal	Which axis is measured and in what units?
Processing	How are offset, RMS and peak calculated?
Decision	Which limit and comparison rule are active?
Validity	What makes a reading unsuitable for scoring?
Evidence	Which file records the demonstrated run?

Appendix C Glossary and configuration terminology

Consistent terminology keeps the hardware description, firmware and evaluation aligned. A sample is one acquired value, whereas a window is the group used to calculate a feature. A trial is an independently labelled experimental run. Confusing these terms can lead to an incorrect denominator or an exaggerated impression of the amount of independent evidence.

A configuration record binds the terms to the actual implementation. It should identify the controller variant, selected channels, feature units, timing, filtering and decision settings. When a meaningful setting changes, use a new identifier and retain the earlier record. This allows a warning to be interpreted even after later software revisions.

The glossary describes the report's usage. It does not establish calibration, completed testing or implementation of every proposed check. Those claims depend on the corresponding measurement record, source code and validation evidence.

Table 31. Glossary of report terms

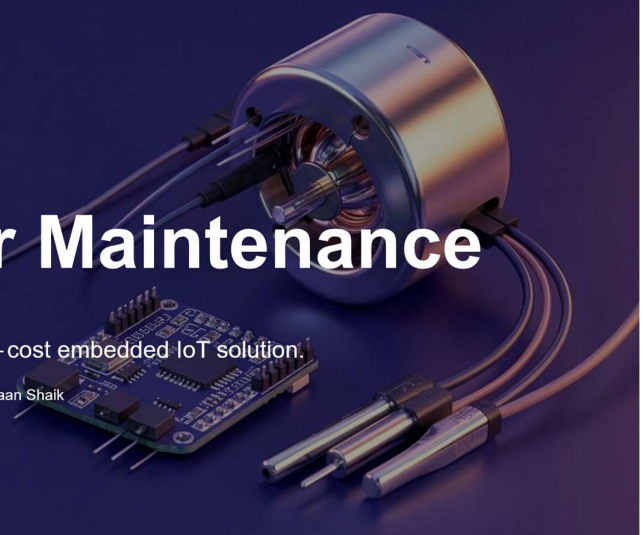
Term	Meaning in this project
ADC	Analog-to-digital converter used to acquire a sensor voltage.
Axis	A defined direction measured by the accelerometer.
Baseline	Reference observations under a stated acceptable condition.
Centred sample	Sample after removal of the window mean.
Feature	Statistic calculated from a sample window.
RMS	Root mean square of the centred samples.
Peak	Largest absolute centred sample in the window.
Threshold	Configured comparison limit in the feature units.
Hysteresis	Different entry and recovery limits.
Persistence	Requirement for a condition to continue across windows.
Validity	Whether data can support the intended decision.
Trial	Independent labelled experimental run.
Detection rate	Detected changed-condition decisions divided by all such labelled decisions.
False-alarm rate	Warnings on normal decisions divided by all labelled normal decisions.

EMBEDDED SYSTEMS · PROJECT REVIEW

ESP32-Based Predictive Motor Maintenance

Vibration analysis and anomaly detection for a low-cost embedded IoT solution.

2510030354 – Mohammad Reyhan · 2510030071 – Faizan · 2510030428 – Amaan Shaik



Made with GAMMA

01 · THE PROBLEM

Motor failures are costly because faults hide in plain sight

- Unexpected failures increase maintenance cost and production downtime.
- Reactive maintenance responds after failure; predictive maintenance looks for warning signals earlier.
- Vibration analysis can reveal abnormal motor behavior before serious damage.



02 · THE PROTOTYPE

The prototype turns vibration into an actionable motor signal

A practical monitoring loop, with implementation status kept explicit.

Reactive maintenance

Responds after failure, increasing maintenance cost and production downtime.

Predictive monitoring

Monitors vibration, compares a baseline or threshold, and looks for warning signals earlier.

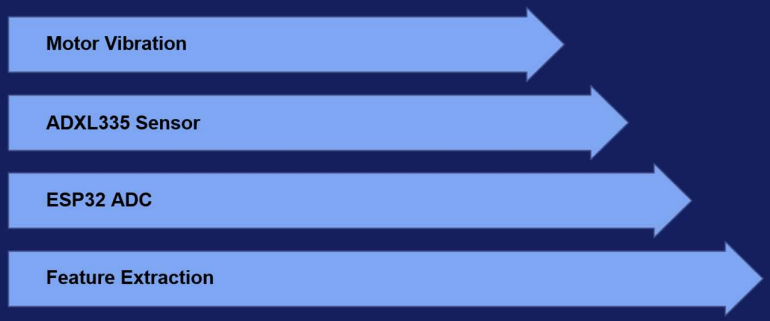
Display monitoring information and alerts only where the implementation supports them.

Made with GAMMA

03 · SYSTEM ARCHITECTURE

From motor movement to a condition estimate

A left-to-right signal path from physical vibration to status.



04 · HARDWARE

Six hardware elements suppo the monitoring loop



Component	Purpose	System role
ESP32 development board	Sensor acquisition and embedded processing	Controller
ADXL335 accelerometer	Three-axis analog vibration measurement	Sensor
Electric motor	Equipment under observation	Observed asset
Wires, breadboard, or connectors	Electrical interconnection	Integration
USB cable and power supply	Programming and power	Support
Optional computer	Serial readings and waveform plots	Visualization

Made with GAMMA

05 · SOFTWARE

The software stack stays embedded and practical

Keep the current claim grounded in what the prototype actually supports.

IMPLEMENTED CORE

Embedded and observable

- ESP32
- Arduino IDE
- Embedded C/C++
- ESP32 ADC
- Serial Monitor
- Serial Plotter

CONDITIONAL / FUTURE

Only when included

- Wi-Fi, HTTP, MQTT, or a dashboard only if included in the actual build.
- Cloud monitoring or machine learning are future enhancement, not part of the current claim.

06 · DETECTION LOGIC

Threshold-based detection makes the algorithm explainable

This is a transparent rule-based method, not machine learning.

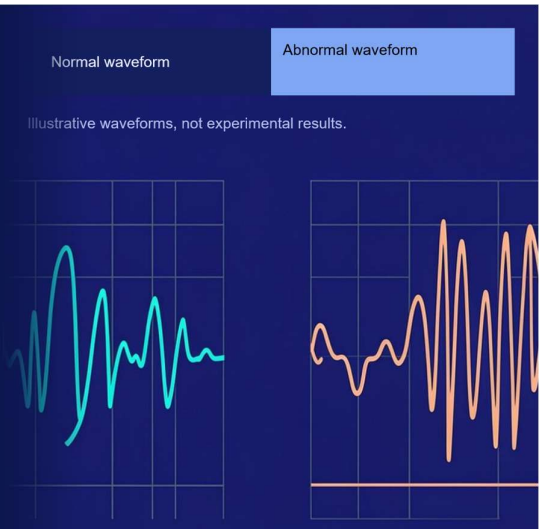


Made with GAMMA

07 · DEMONSTRATION

The demonstration connects the sensor, signal, and status

- Secure the vibration sensor to the motor and connect it to the ESP32.
- Read analog signals and observe them in the Serial Monitor or Serial Plotter.
- Compare normal and abnormal behavior using clearly labeled illustrative waveforms.
- Use sample code or pseudocode only where it improves explanation.



Illustrative waveforms, not experimental results.

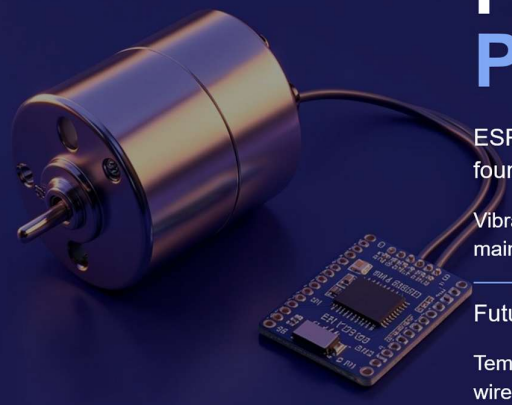
08 · EVALUATION

Evaluation should separate evidence from expected valu

Category	Evaluation
Evaluate	RMS vibration, peak amplitude, detection rate, false alarms, and response time.
Expected outcome	Detect amplitude changes and threshold exceedances in real time.
Benefits	Earlier fault indication, lower -cost monitoring, and better maintenance planning.
Applications	Motors and pumps, manufacturing equipment, HVAC, conveyors, and industrial automation.

Do not invent accuracy percentages or verified findings.

Made with GAMMA



09 · CLOSING

Predict Today, Prevent Tomorrow

ESP32 plus vibration sensing provides a practical foundation for motor condition monitoring.

Vibration -based anomaly detection can support preventive maintenance decisions.

Future scope

Temperature sensing, better filtering and feature extraction, wireless dashboards and notifications, condition -specific calibration, and machine -learning extensions.

Thank You · Mohammad Reyhan · Faizan · Amaan Shaik

Made with GAMMA